2116251901128
PRAVINTHEKAA R.L

COMPUTER NETWORKS

Ex.No.7	**Write a code using RAW sockets to implement packet sniffing.**
DATE:	

AIM:

To develop a Python program using raw sockets that captures Ethernet frames and extracts the source and destination MAC addresses and protocol details for network analysis and troubleshooting.

Definition:

Packet sniffing is the technique of capturing and examining network packets transmitted over a network to monitor communication, troubleshoot network issues, and analyze network traffic.

Algorithm:

1. Start.
2. Import the required modules.
3. Get the local host IP address.
4. Create and bind a raw socket.
5. Enable IP header inclusion and promiscuous mode.
6. Set the packet count to **5**.
7. Receive one packet at a time.
8. Extract and display:
   Destination MAC address
   Source MAC address
   Protocol type
   Packet length
9. Repeat until **5 packets** are captured.
10. Disable promiscuous mode.
11. Display the completion message.
12. Stop.

PROGRAM:

```
import socket
import struct

def get_mac_addr(bytes_addr):
```

```
 """Convert MAC address from bytes to readable format."""
return ':'.join('{:02X}'.format(b) for b in bytes_addr)

def get_protocol(bytes_proto):
 """Convert protocol bytes to hexadecimal."""
 return hex(struct.unpack('!H', bytes_proto)[0])

def ethernet_frame(data):
 """Extract Ethernet frame information."""
 dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14])
return (
 get_mac_addr(dest_mac),
 get_mac_addr(src_mac),
 get_protocol(proto),
 data[14:]
 )
def main():
 host = socket.gethostbyname(socket.gethostname())
print("Listening on:", host)
 try:
 # Create raw socket
 conn = socket.socket(socket.AF_INET,
 socket.SOCK_RAW,
 socket.IPPROTO_IP)

 # Bind socket
 conn.bind((host, 0))

 # Include IP header
 conn.setsockopt(socket.IPPROTO_IP,
 socket.IP_HDRINCL,
 1)

 # Enable promiscuous mode
 conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print("\nPacket Sniffer Started...\n")

 # Capture only 5 packets
 packet_count = 5
 for i in range(packet_count):

 raw_data, addr = conn.recvfrom(65536)
```

```
        print(f"\nPacket {i+1}")
        print("-" * 40)

        if len(raw_data) >= 14:
        dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)

        print("Destination MAC :", dest_mac)
        print("Source MAC :", src_mac)
        print("Protocol :", eth_proto)
        print("Packet Length :", len(raw_data), "bytes")

        # Disable promiscuous mode
        conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)

       print("\nPacket Sniffer Stopped Successfully.")

        except PermissionError:
        print("Run this program as Administrator.")

        except KeyboardInterrupt:
        conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
       print("\nPacket Sniffer Stopped by User.")

       if __name__ == "__main__":
        main()
```

SAMPLE OUTPUT:

```
Listening on: 192.168.1.10

Packet Sniffer Started...

Packet 1
----------------------------------------
Destination MAC : 45:00:00:3C:1C:46
Source MAC : 40:06:B1:E6:C0:A8
Protocol : 0x101
Packet Length : 60 bytes

Packet 2
----------------------------------------
Destination MAC : 45:00:00:34:2F:9A
Source MAC : 40:11:C3:D8:C0:A8
Protocol : 0x101
Packet Length : 52 bytes
```

Packet 3

Destination MAC : 45:00:00:54:3B:8F
Source MAC : 40:06:91:AB:C0:A8
Protocol : 0x101
Packet Length : 84 bytes

Packet 4

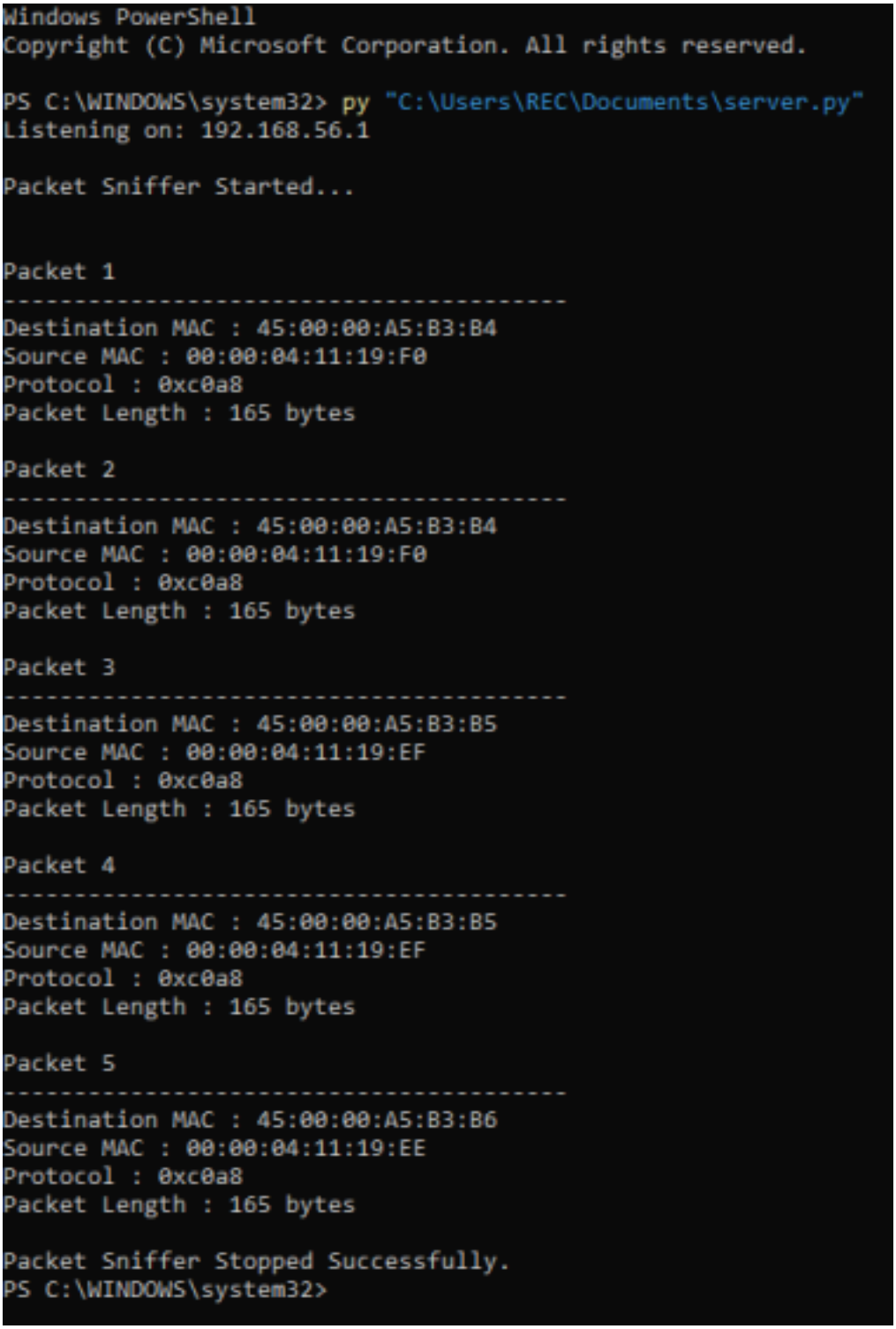
Destination MAC : 45:00:00:28:5D:71
Source MAC : 40:01:6F:12:C0:A8
Protocol : 0x101
Packet Length : 40 bytes

Packet 5

Destination MAC : 45:00:00:3C:7A:15
Source MAC : 40:06:5A:9F:C0:A8
Protocol : 0x101
Packet Length : 60 bytes

Packet Sniffer Stopped Successfully.

OUTPUT:



RESULT:

Therefore raw code has been used to implement packet sniffing has been verified.